



C Piscine

C 12

Preâmbulo: Este documento é o subject para o módulo C 12 da Piscine C na 42.

Versão: 9.0

Sumário

I	Prefácio	2
II	Instruções	4
III	Instruções de IA	5
IV	Exercício 00 : ft_create_elem	8
V	Exercício 01 : ft_list_push_front	9
VI	Exercício 02 : ft_list_size	10
VII	Exercício 03 : ft_list_last	11
VIII	Exercício 04 : ft_list_push_back	12
IX	Exercício 05 : ft_list_push_strs	13
X	Exercício 06 : ft_list_clear	14
XI	Exercício 07 : ft_list_at	15
XII	Exercício 08 : ft_list_reverse	16
XIII	Exercício 09 : ft_list_foreach	17
XIV	Exercício 10 : ft_list_foreach_if	18
XV	Exercício 11 : ft_list_find	19
XVI	Exercício 12 : ft_list_remove_if	20
XVII	Exercício 13 : ft_list_merge	21
XVIII	Exercício 14 : ft_list_sort	22
XIX	Exercício 15 : ft_list_reverse_fun	23
XX	Exercício 16 : ft_sorted_list_insert	24
XXI	Exercício 17 : ft_sorted_list_merge	25
XXII	Entrega e avaliação por pares	26

Capítulo I

Prefácio

ALERTA DE SPOILER
NÃO LEIA A PRÓXIMA PÁGINA

Você foi avisado.

- Em Star Wars, Darth Vader é o pai de Luke.
- Em Os Suspeitos, Verbal é Keyser Soze.
- Em Clube da Luta, Tyler Durden e o narrador são a mesma pessoa.
- Em O Sexto Sentido, Bruce Willis está morto desde o começo.
- Em Os Outros, os habitantes da casa são fantasmas e vice-versa.
- Em Bambi, a mãe de Bambi morre.
- Em A Vila, os monstros são os aldeões e o filme se passa nos nossos tempos.
- Em Harry Potter, Dumbledore morre.
- Em Planeta dos Macacos, o filme se passa na Terra.
- Em Game of Thrones, Robb Stark e Joffrey Baratheon morrem no dia de seus casamentos.
- Em Crepúsculo, vampiros brilham sob o sol.
- Em Stargate SG-1, Temporada 1, Episódio 18, O'Neill e Carter estão na Antártica.
- Em O Cavaleiro das Trevas Ressurge, Miranda Tate é Talia Al'Gul.
- Em Super Mario Bros, a princesa está em outro castelo.

Capítulo II

Instruções

- Você só pode usar as funções autorizadas em cada exercício.
- Seu código deve respeitar os padrões C. Ele deve compilar com `gcc` usando as flags `-Wall -Wextra -Werror`.
- O trabalho deve ser entregue nos arquivos especificados.
- Suas funções não devem vaziar memória.
- Variáveis globais não são permitidas.
- Você não tem permissão para usar quaisquer funções de biblioteca a menos que explicitamente declarado.
- Todos os includes devem estar dentro do arquivo `.c`.
- A função `main` para teste deve estar em um arquivo separado (não submetido).
- Para os exercícios a seguir, você deve usar a seguinte estrutura:

```
typedef struct          s_list
{
    struct s_list      *next;
    void               *data;
}                      t_list;
```

- Você terá que incluir esta estrutura em um arquivo `ft_list.h` e submetê-lo para cada exercício.
- A partir do exercício 01 em diante, usaremos nossa `ft_create_elem`, então faça os arranjos necessários (poderia ser útil ter seu protótipo em um arquivo `ft_list.h...`).

Capítulo III

Instruções de IA

● Contexto

A C Piscine é intensa. É seu primeiro grande desafio na 42 — um mergulho profundo na resolução de problemas, autonomia e comunidade.

Durante essa fase, seu objetivo principal é construir sua base — através do esforço, da repetição e, especialmente, da troca de **aprendizagem entre pares**.

Na era da IA, atalhos são fáceis de encontrar. No entanto, é importante considerar se o uso da IA está realmente ajudando você a crescer — ou simplesmente atrapalhando o desenvolvimento de habilidades reais.

A Piscine também é uma experiência humana — e, por enquanto, nada pode substituí-la. Nem mesmo a IA.

Para uma visão mais completa de nossa posição sobre a IA — como ferramenta de aprendizagem, como parte do currículo de TIC e como uma expectativa crescente no mercado de trabalho — consulte as perguntas frequentes disponíveis na intranet.

● Mensagem principal

- ✎ Construa bases sólidas sem atalhos.
- ✎ Desenvolva realmente habilidades técnicas e de poder.
- ✎ Experimente a verdadeira aprendizagem entre pares, comece a aprender como aprender e resolver novos problemas.
- ✎ A jornada de aprendizagem é mais importante que o resultado.
- ✎ Aprenda sobre os riscos associados à IA e desenvolva práticas de controle eficazes e contramedidas para evitar armadilhas comuns.

● Regras para o aluno:

- Você deve aplicar o raciocínio às tarefas atribuídas, especialmente antes de recorrer à IA.
- Você não deve pedir respostas diretas à IA.
- Você deve aprender sobre a abordagem global da 42 em relação à IA.

● Resultados da fase:

Dentro desta fase fundamental, você obterá os seguintes resultados:

- Obter bases adequadas em tecnologia e codificação.
- Saber por que e como a IA pode ser perigosa durante esta fase.

● Comentários e exemplo:

- Sim, sabemos que a IA existe — e sim, ela pode resolver seus projetos. Mas você está aqui para aprender, não para provar que a IA aprendeu. Não perca seu tempo (nem o nosso) apenas para demonstrar que a IA pode resolver o problema proposto.
- Aprender na 42 não é sobre saber a resposta — é sobre desenvolver a capacidade de encontrá-la. A IA lhe dá a resposta diretamente, mas isso impede você de construir seu próprio raciocínio. E o raciocínio leva tempo, esforço e envolve falhas. O caminho para o sucesso não deve ser fácil.
- Lembre-se de que durante os exames, a IA não está disponível — sem internet, sem smartphones, etc. Você perceberá rapidamente se confiou muito na IA em seu processo de aprendizagem.
- A aprendizagem entre pares o expõe a diferentes ideias e abordagens, melhorando suas habilidades interpessoais e sua capacidade de pensar de forma divergente. Isso é muito mais valioso do que apenas conversar com um bot. Portanto, não seja tímido — converse, faça perguntas e aprenda juntos!
- Sim, a IA fará parte do currículo — tanto como ferramenta de aprendizagem quanto como tópico em si. Você terá até a chance de construir seu próprio software de IA. Para saber mais sobre nossa abordagem crescente, consulte a documentação disponível na intranet.

✓ Boa prática:

Estou travado em um novo conceito. Pergunto a alguém próximo como ele abordou isso. Conversamos por 10 minutos — e de repente, clica. Entendi.

✗ Má prática:

Usei secretamente a IA, copiei um código que parece certo. Durante a avaliação entre pares, não consigo explicar nada. Eu falho. Durante o exame — sem IA — estou travado novamente. Eu falho.

Capítulo IV

Exercício 00 : ft_create_elem

	Exercício : 00
	ft_create_elem
	Pasta de entrega : <i>ex00/</i>
	Arquivos para entregar : <code>ft_create_elem.c</code> , <code>ft_list.h</code>
	Funções ou bibliotecas autorizadas : <code>malloc</code>

- Crie a função `ft_create_elem`, que cria um novo elemento do tipo `t_list`.
- Ela deve atribuir `data` ao argumento fornecido e `next` a `NULL`.
- Aqui está como ela deve ser prototipada:

```
t_list *ft_create_elem(void *data);
```

Capítulo V

Exercício 01 : ft_list_push_front

	Exercício : 01
	ft_list_push_front
	Pasta de entrega : <i>ex01/</i>
	Arquivos para entregar : <code>ft_list_push_front.c</code> , <code>ft_list.h</code>
	Funções ou bibliotecas autorizadas : <code>ft_create_elem</code>

- Crie a função `ft_list_push_front`, que adiciona um novo elemento do tipo `t_list` ao início da lista.
- Ela deve atribuir `data` ao argumento fornecido.
- Se necessário, ela atualizará o ponteiro para o início da lista.
- Aqui está como ela deve ser prototipada:

```
void      ft_list_push_front(t_list **begin_list, void *data);
```

Capítulo VI

Exercício 02 : ft_list_size

	Exercício : 02
	ft_list_size
	Pasta de entrega : <i>ex02/</i>
	Arquivos para entregar : <code>ft_list_size.c</code> , <code>ft_list.h</code>
	Funções ou bibliotecas autorizadas : Nenhuma

- Crie a função `ft_list_size`, que retorna o número de elementos na lista.
- Aqui está como ela deve ser prototipada:

```
int ft_list_size(t_list *begin_list);
```

Capítulo VII

Exercício 03 : ft_list_last

	Exercício : 03
	ft_list_last
	Pasta de entrega : <i>ex03/</i>
	Arquivos para entregar : <code>ft_list_last.c</code> , <code>ft_list.h</code>
	Funções ou bibliotecas autorizadas : Nenhuma

- Crie a função `ft_list_last`, que retorna o último elemento da lista.
- Aqui está como ela deve ser prototipada:

```
t_list *ft_list_last(t_list *begin_list);
```

Capítulo VIII

Exercício 04 : ft_list_push_back

	Exercício : 04
	ft_list_push_back
	Pasta de entrega : <i>ex04/</i>
	Arquivos para entregar : <code>ft_list_push_back.c</code> , <code>ft_list.h</code>
	Funções ou bibliotecas autorizadas : <code>ft_create_elem</code>

- Crie a função `ft_list_push_back`, que adiciona um novo elemento do tipo `t_list` ao final da lista.
- Ela deve atribuir `data` ao argumento fornecido.
- Se necessário, ela atualizará o ponteiro para o início da lista.
- Aqui está como ela deve ser prototipada:

```
void      ft_list_push_back(t_list **begin_list, void *data);
```

Capítulo IX

Exercício 05 : ft_list_push_strs

	Exercício : 05
	ft_list_push_strs
	Pasta de entrega : <i>ex05/</i>
	Arquivos para entregar : <code>ft_list_push_strs.c</code> , <code>ft_list.h</code>
	Funções ou bibliotecas autorizadas : <code>ft_create_elem</code>

- Crie a função `ft_list_push_strs`, que cria uma nova lista que inclui todas as strings apontadas pelos elementos em `strs`.
- `size` é o tamanho de `strs`.
- O primeiro elemento deve estar no final da lista.
- O endereço do primeiro link na lista é retornado.
- Aqui está como ela deve ser prototipada:

```
t_list *ft_list_push_strs(int size, char **strs);
```

Capítulo X

Exercício 06 : ft_list_clear

	Exercício : 06
	ft_list_clear
	Pasta de entrega : <i>ex06/</i>
	Arquivos para entregar : <code>ft_list_clear.c</code> , <code>ft_list.h</code>
	Funções ou bibliotecas autorizadas : free

- Crie a função `ft_list_clear`, que remove e libera todos os links da lista.
- `free_fct` é usada para liberar cada `data`.
- Aqui está como ela deve ser prototipada:

```
void ft_list_clear(t_list *begin_list, void (*free_fct)(void *));
```

Capítulo XI

Exercício 07 : ft_list_at

	Exercício : 07
ft_list_at	
Pasta de entrega : <i>ex07/</i>	
Arquivos para entregar : <i>ft_list_at.c, ft_list.h</i>	
Funções ou bibliotecas autorizadas : <i>Nenhuma</i>	

- Crie a função `ft_list_at`, que retorna o N-ésimo elemento da lista, sabendo que o primeiro elemento da lista é quando `nbr` é igual a 0.
- Em caso de erro, ela deve retornar um ponteiro nulo.
- Aqui está como ela deve ser prototipada:

```
t_list *ft_list_at(t_list *begin_list, unsigned int nbr);
```


Capítulo XII

Exercício 08 : ft_list_reverse

	Exercício : 08
	ft_list_reverse
	Pasta de entrega : <i>ex08/</i>
	Arquivos para entregar : <code>ft_list_reverse.c</code>
	Funções ou bibliotecas autorizadas : Nenhuma

- Crie a função `ft_list_reverse`, que inverte a ordem dos elementos de uma lista. O valor de cada elemento deve permanecer o mesmo.
- Observe que nesta função, usaremos nosso próprio `ft_list.h`.
- Aqui está como ela deve ser prototipada:

```
void ft_list_reverse(t_list **begin_list);
```

Capítulo XIII

Exercício 09 : ft_list_foreach

	Exercício : 09
	ft_list_foreach
	Pasta de entrega : <i>ex09/</i>
	Arquivos para entregar : <i>ft_list_foreach.c, ft_list.h</i>
	Funções ou bibliotecas autorizadas : <i>Nenhuma</i>

- Crie a função `ft_list_foreach`, que aplica a função dada como argumento a cada um dos elementos da lista.
- `f` deve ser aplicada na mesma ordem da lista.
- Aqui está como ela deve ser prototipada:

```
void ft_list_foreach(t_list *begin_list, void (*f)(void *));
```

- A função apontada por `f` será usada da seguinte forma:

```
(*f)(list_ptr->data);
```

Capítulo XIV

Exercício 10 : ft_list_foreach_if

	Exercício : 10
	ft_list_foreach_if
	Pasta de entrega : <i>ex10/</i>
	Arquivos para entregar : <i>ft_list_foreach_if.c, ft_list.h</i>
	Funções ou bibliotecas autorizadas : Nenhuma

- Crie a função `ft_list_foreach_if`, que aplica a função dada como argumento a alguns dos elementos da lista.
- Aplique a função apenas aos elementos quando `cmp` com `data_ref` retornar 0.
- `f` deve ser aplicada na mesma ordem da lista.
- Aqui está como ela deve ser prototipada:

```
void ft_list_foreach_if(t_list *begin_list, void (*f)(void *), void  
*data_ref, int (*cmp)());
```

- As funções apontadas por `f` e por `cmp` serão usadas da seguinte forma:

```
(*f)(list_ptr->data);  
(*cmp)(list_ptr->data, data_ref);
```



Por exemplo, a função `cmp` poderia ser `ft_strcmp...`

Capítulo XV

Exercício 11 : ft_list_find

	Exercício : 11
	ft_list_find
	Pasta de entrega : <i>ex11/</i>
	Arquivos para entregar : <code>ft_list_find.c</code> , <code>ft_list.h</code>
	Funções ou bibliotecas autorizadas : Nenhuma

- Crie a função `ft_list_find` que retorna o endereço dos dados do primeiro elemento onde compará-lo com `data_ref` usando `cmp` faz com que `cmp` retorne 0.
- Aqui está como ela deve ser prototipada:

```
t_list *ft_list_find(t_list *begin_list, void *data_ref, int (*cmp)());
```

- A função apontada por `cmp` será usada da seguinte forma:

```
(*cmp)(list_ptr->data, data_ref);
```

Capítulo XVI

Exercício 12 : ft_list_remove_if

	Exercício : 12
	ft_list_remove_if
	Pasta de entrega : <i>ex12/</i>
	Arquivos para entregar : <code>ft_list_remove_if.c</code> , <code>ft_list.h</code>
	Funções ou bibliotecas autorizadas : <code>free</code>

- Crie a função `ft_list_remove_if` que remove da lista todos os elementos cujos dados, quando comparados a `data_ref` usando `cmp`, fazem com que `cmp` retorne 0.
- Os dados de um elemento a ser apagado devem ser liberados usando `free_fct`.
- Aqui está como ela deve ser prototipada:

```
void ft_list_remove_if(t_list **begin_list, void *data_ref, int (*cmp)(), void (*free_fct)(void *))
```

- As funções apontadas por `cmp` e `free_fct` serão usadas da seguinte forma:

```
(*cmp)(list_ptr->data, data_ref);  
(*free_fct)(list_ptr->data);
```

Capítulo XVII

Exercício 13 : ft_list_merge

	Exercício : 13
ft_list_merge	
Pasta de entrega : <i>ex13/</i>	
Arquivos para entregar : <i>ft_list_merge.c, ft_list.h</i>	
Funções ou bibliotecas autorizadas : Nenhuma	

- Crie a função `ft_list_merge` que coloca os elementos de uma lista `begin2` no final de outra lista `begin1`.
- A criação de elementos não é autorizada.
- Aqui está como ela deve ser prototipada:

```
void ft_list_merge(t_list **begin_list1, t_list *begin_list2);
```

Capítulo XVIII

Exercício 14 : ft_list_sort

	Exercício : 14
	ft_list_sort
	Pasta de entrega : <i>ex14/</i>
	Arquivos para entregar : <code>ft_list_sort.c</code> , <code>ft_list.h</code>
	Funções ou bibliotecas autorizadas : Nenhuma

- Crie a função `ft_list_sort` que ordena os elementos da lista em ordem crescente, comparando dois elementos e seus dados usando uma função de comparação.
- Aqui está como ela deve ser prototipada:

```
void ft_list_sort(t_list **begin_list, int (*cmp)());
```

- A função apontada por `cmp` será usada da seguinte forma:

```
(*cmp)(list_ptr->data, list_other_ptr->data);
```



`cmp` poderia ser, por exemplo, `ft_strcmp`.

Capítulo XIX

Exercício 15 : ft_list_reverse_fun

	Exercício : 15
	ft_list_reverse_fun
	Pasta de entrega : <i>ex15/</i>
	Arquivos para entregar : <code>ft_list_reverse_fun.c</code> , <code>ft_list.h</code>
	Funções ou bibliotecas autorizadas : Nenhuma

- Crie a função `ft_list_reverse_fun` que inverte a ordem dos elementos na lista.
- Aqui está como ela deve ser prototipada:

```
void ft_list_reverse_fun(t_list *begin_list);
```


Capítulo XX

Exercício 16 : ft_sorted_list_insert

	Exercício : 16
	ft_sorted_list_insert
	Pasta de entrega : <i>ex16/</i>
	Arquivos para entregar : <code>ft_sorted_list_insert.c</code> , <code>ft_list.h</code>
	Funções ou bibliotecas autorizadas : <code>ft_create_elem</code>

- Crie a função `ft_sorted_list_insert` que cria um novo elemento e o insere em uma lista ordenada de forma que ela permaneça ordenada em ordem crescente.
- Aqui está como ela deve ser prototipada:

```
void ft_sorted_list_insert(t_list **begin_list, void *data, int (*cmp)());
```

- A função apontada por `cmp` será usada da seguinte forma:

```
(*cmp)(list_ptr->data, list_other_ptr->data);
```

Capítulo XXI

Exercício 17 : ft_sorted_list_merge

	Exercício : 17
ft_sorted_list_merge	
Pasta de entrega : <i>ex17/</i>	
Arquivos para entregar : <i>ft_sorted_list_merge.c, ft_list.h</i>	
Funções ou bibliotecas autorizadas : Nenhuma	

- Crie a função `ft_sorted_list_merge` que integra os elementos de uma lista ordenada `begin2` em outra lista ordenada `begin1`, de forma que `begin1` permaneça ordenada em ordem crescente.
- Aqui está como ela deve ser prototipada:

```
void ft_sorted_list_merge(t_list **begin_list1, t_list *begin_list2, int (*cmp)());
```

- A função apontada por `cmp` será usada da seguinte forma:

```
(*cmp)(list_ptr->data, list_other_ptr->data);
```

Capítulo XXII

Entrega e avaliação por pares

Envie sua tarefa para o seu repositório `Git` como de costume. Apenas o trabalho dentro do seu repositório será avaliado durante a defesa. Certifique-se de verificar novamente os nomes dos arquivos para garantir que estão corretos.



Você deve enviar apenas os arquivos explicitamente exigidos pelas instruções do projeto.